

Embedded Programming

Dr. Charis Kouzinopoulos

Department of Advanced Computing Sciences

Maastricht University

2024-2025

charis.kouzinopoulos@maastrichtuniversity.nl



Maastricht University

ARM assembly

Procedures / functions

Procedures / functions / methods

- A procedure (or function) is one tool programmers use to structure programs, both to make them easier to understand and to allow code to be reused
- You can think of a procedure like a **spy** who leaves with a secret plan, acquires resources, performs the task, covers his or her tracks, and then returns to the point of origin with the desired result. Nothing else should be perturbed once the mission is complete.



History

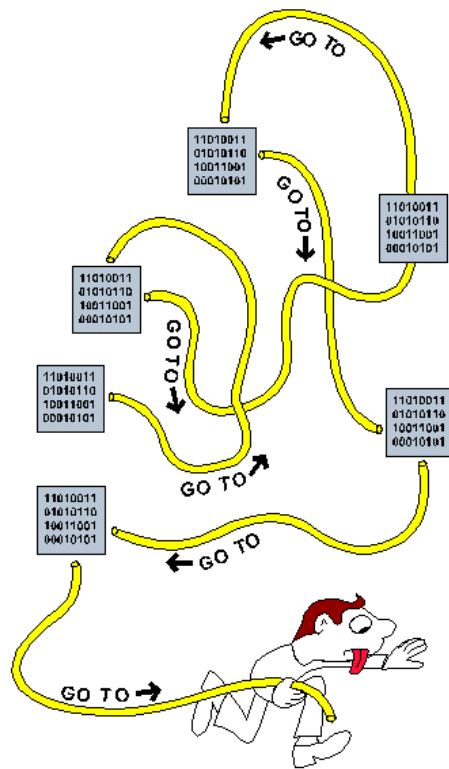
- Spaghetti Code
 - Before the 1980s, program flow bounces around anywhere the programmer wanted
 - Culprit: overusing "GOTO" statements

Spaghetti code in BASIC

```
1 i=0
2 i=i+1
3 PRINT i; "squared=";i*i
4 IF i>=100 THEN GOTO 6
5 GOTO 2
6 PRINT "Program Completed."
7 END
```

Structured programming in BASIC

```
1 FOR i=1 TO 100
2   PRINT i;"squared=";i*i
3 NEXT i 'termination of loop body
4 PRINT "Program Completed."
5 END
```



Importance of Structured Programming

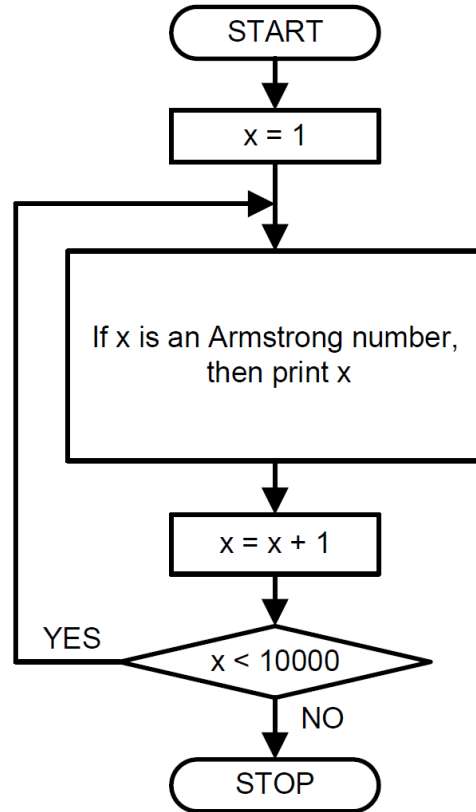
- Assembly is not a structured programming language
 - Does not directly support selection and loop
 - Branch in assembly = “goto” in C
 - Break the single-entry single-exit rule
- Easy to generate spaghetti code in assembly
 - Twisted and tangled
 - Difficult to debug & maintain
- One strategy to alleviate the challenge
 - Use **flowcharts** to facilitate assembly programming



© dreamstime.com

© 13899988 - Pngtree

Top-Down Design Example

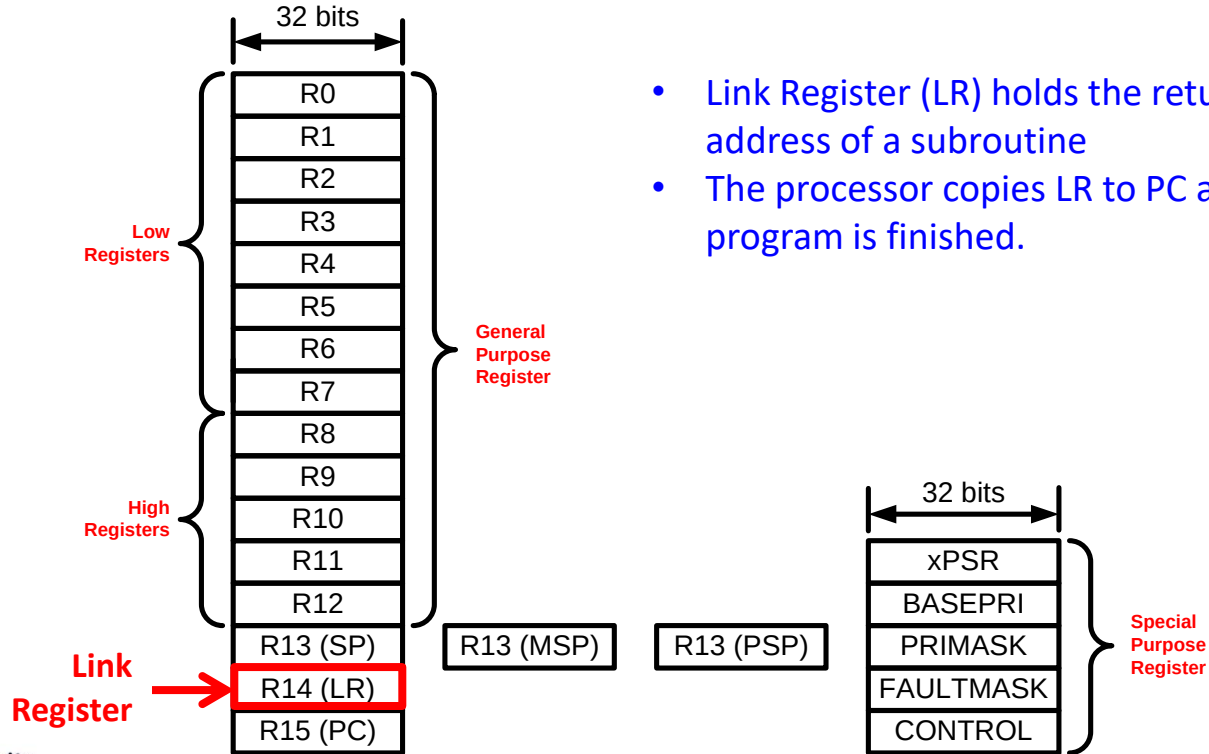


Procedures / functions / methods

In the execution of a procedure, the program must follow these six steps:

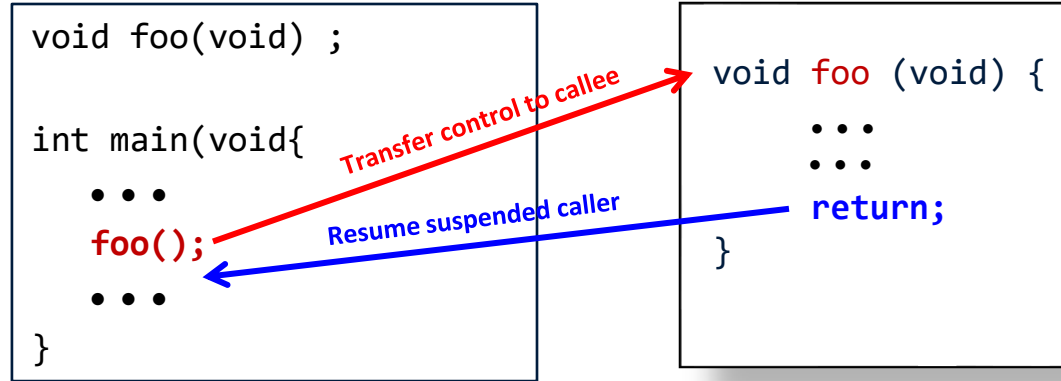
1. Put parameters in a place where the procedure can access them
2. Transfer control to the procedure
3. Acquire the storage resources needed for the procedure
4. Perform the desired task
5. Put the result value in a place where the calling program can access it
6. Return control to the point of origin, since a procedure can be called from several points in a program

Link Register (LR)

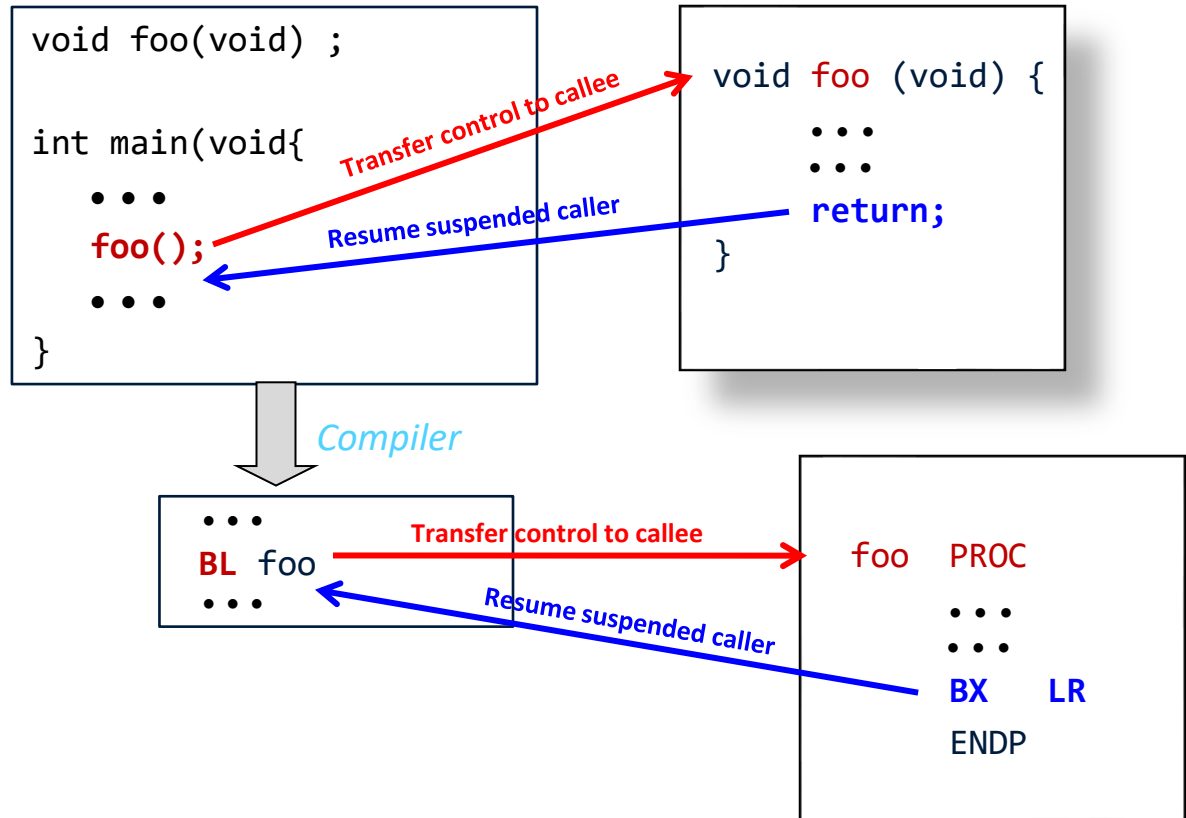


- Link Register (LR) holds the return address of a subroutine
- The processor copies LR to PC after the program is finished.

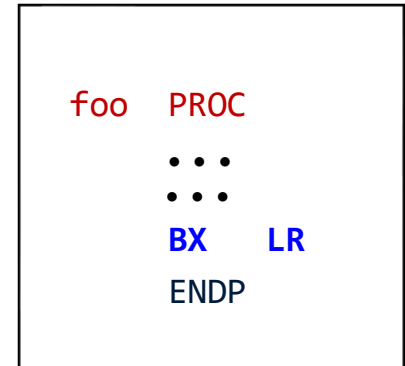
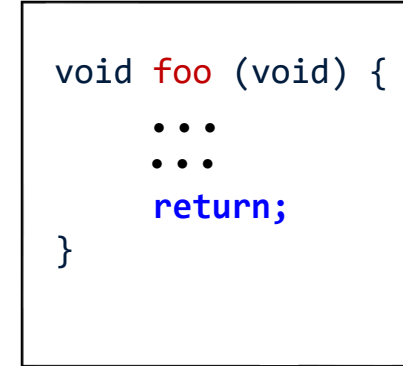
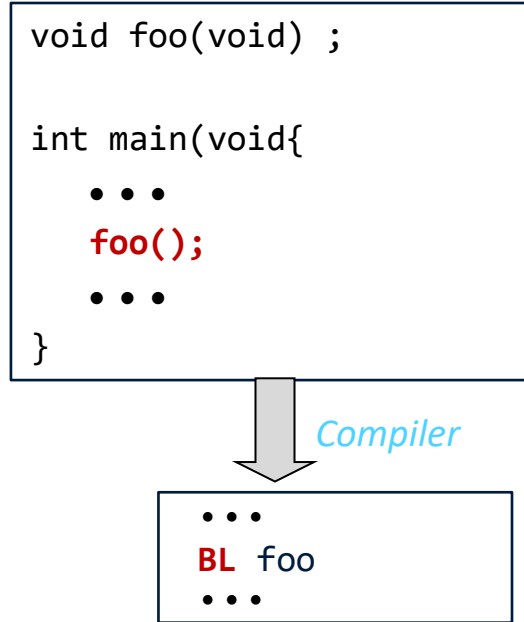
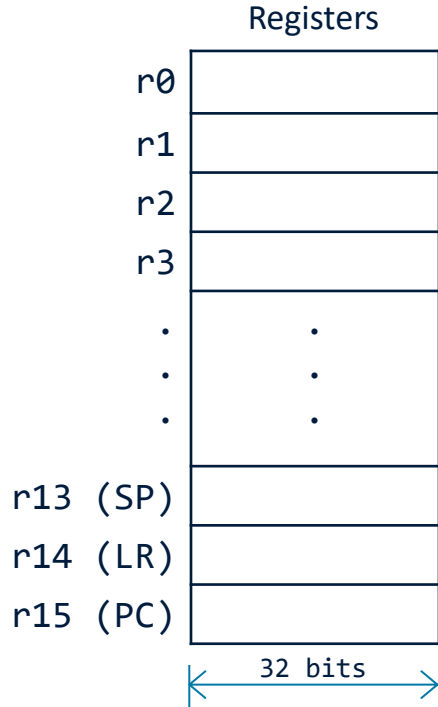
Link Register (LR)



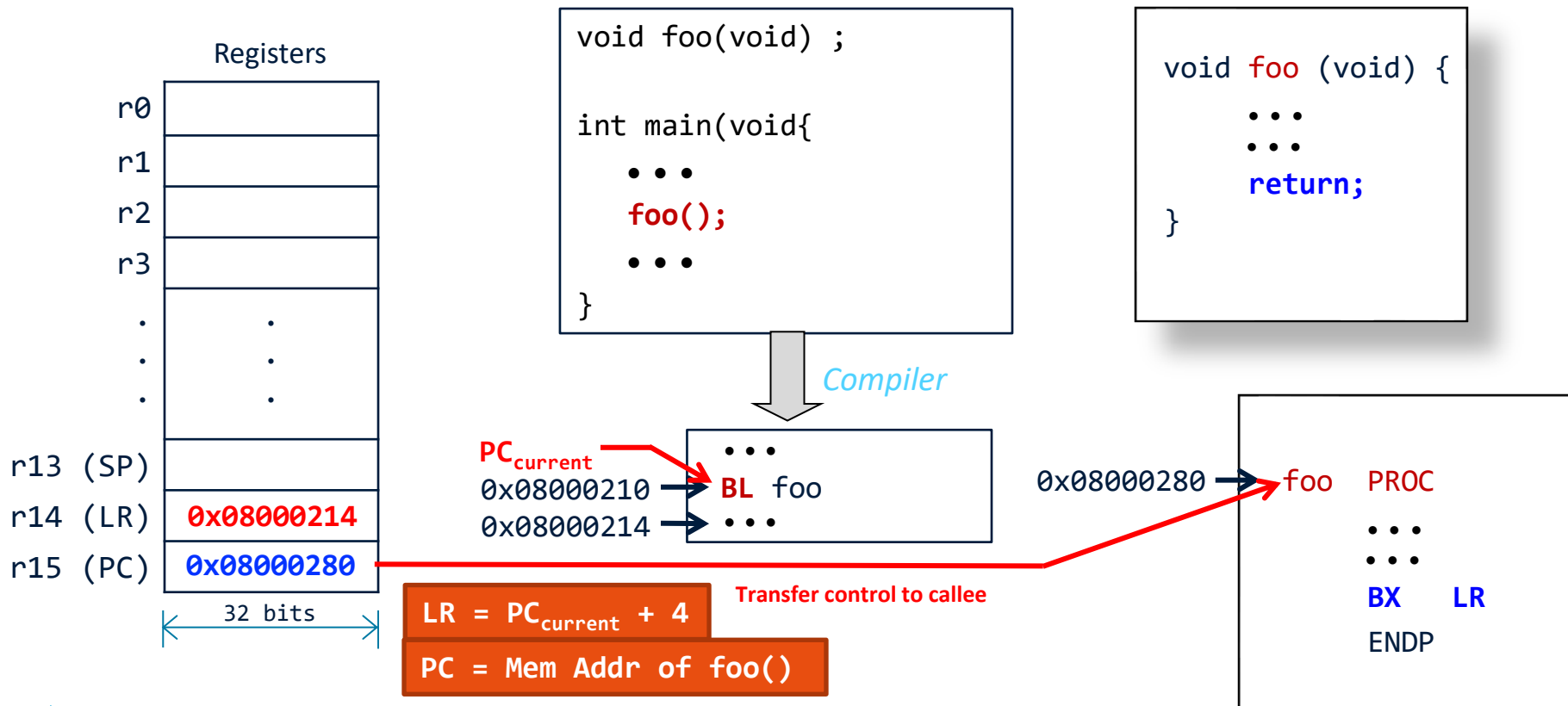
Link Register (LR)



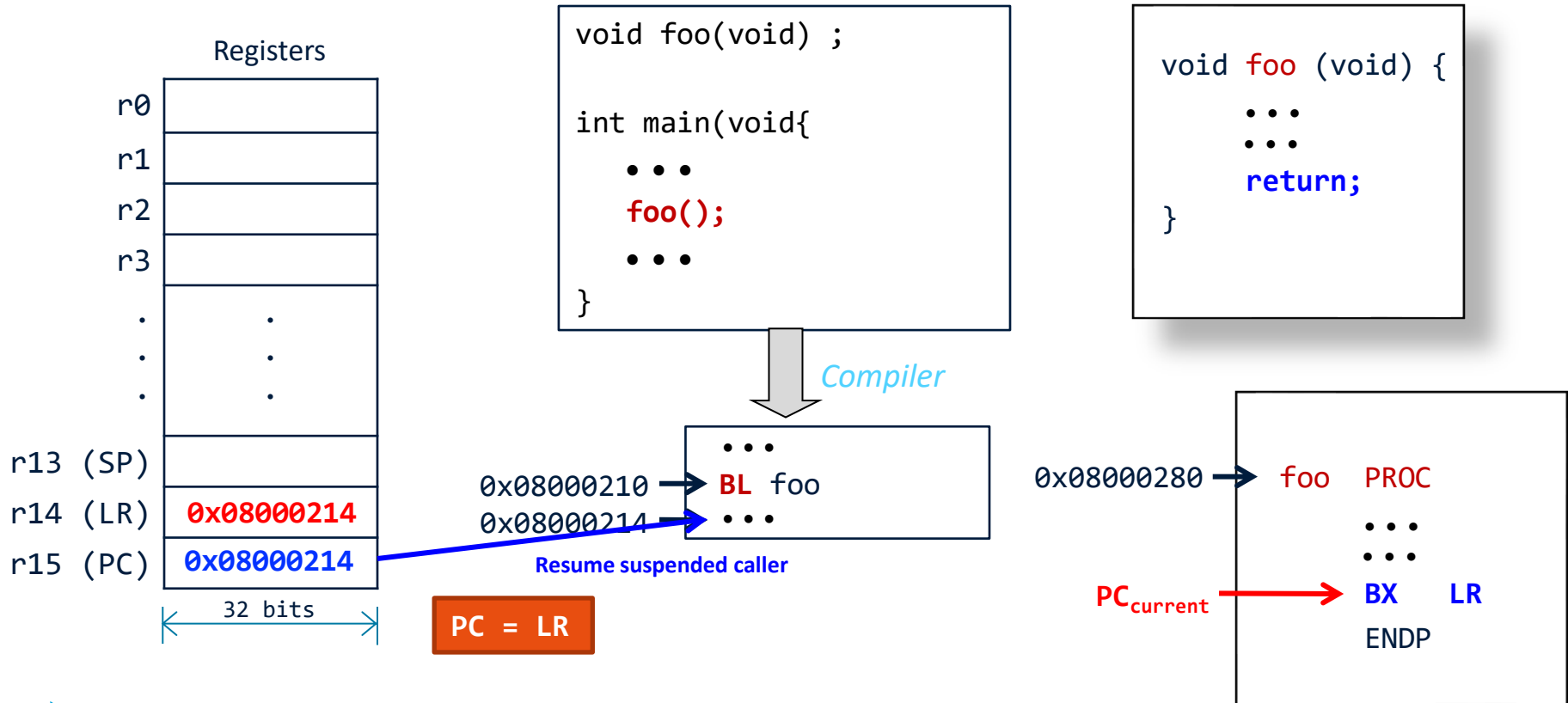
Link Register (LR)



Link Register (LR)



Link Register (LR)



Procedures / functions / methods - BL

- Registers are the fastest place to hold data in a computer, so we want to use them as much as possible. The following convention is followed for procedure calling:
 - R0-R3: four parameter registers in which to pass parameters or return values
 - LR: one return address register to return to the point of origin
- ARM also includes an instruction just for the procedures: it branches to an address and simultaneously saves the address of the following instruction in LR. The *branch-and-link* instruction (BL) is simply written:

BL ProcedureAddress

Procedures / functions / methods - BL

```
int result = add_two_numbers(3, 7);
```

08000332:	movs	r1, #7
08000334:	movs	r0, #3
08000336:	bl	0x8000278 <add_two_numbers>

- The link portion of the name means that an address or link is formed that points to the calling site to allow the procedure to return to the proper address. This “link”, stored in LR, is called the **return address**
- The return address is needed because the same procedure could be called from several parts of the program

Procedures / functions / methods - BR

- To support returning from a procedure, the **branch and exchange** instruction (BX) is used, meaning an unconditional branch to the address specified in a register:

BX LR

- The branch register instruction branches to the address stored in register LR - which is just what we want
- Thus, the calling program, or **caller**, puts the parameter values in registers and uses *BL X* to branch to procedure X - sometimes named the **callee**
- The **callee** then performs the calculations, places the results in the same parameter registers, and returns control to the caller using *BX LR*

Procedures / functions / methods - BR

```
int result = add_two_numbers(3, 7);
```

```
08000278:  
27
```

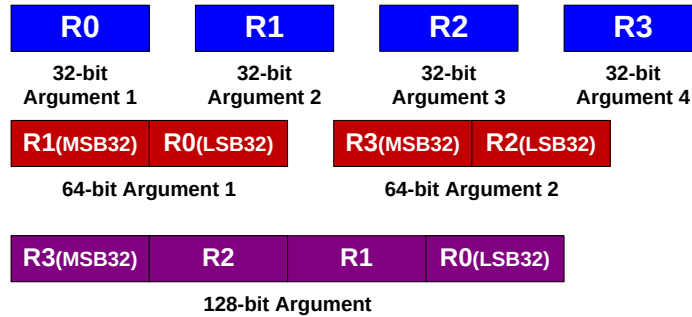
```
add_two_numbers:  
    adds    r0, r0, r1  
    bx lr
```



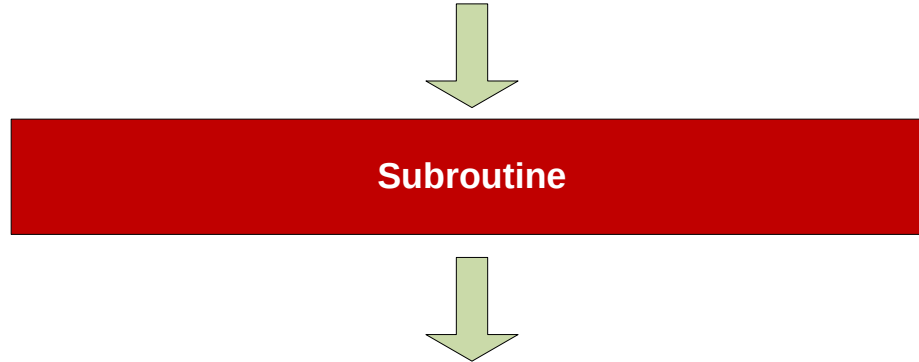
Procedures / functions / methods - PC

- Implicit in the stored-program idea is the need to have a register to hold the address of the current instruction being executed. For historical reasons, this register is almost always called the program counter, abbreviated **PC**
- The BL instruction actually saves PC + 4 in register LR to link to the byte address of the following instruction to set up the procedure return

Passing Arguments and Returning Value



Extra arguments are pushed to the stack by the caller. The caller is responsible to pop them out of the stack after the subroutine returns.



Example

- In *assembler.s* define two new functions, `function1` and `function2` that accept an integer as an argument
- Call them one after the other from `ASM_Function`, making sure that the callee returns every time to the caller
- `function1` should increase the value of the argument by 5
- `function2` should decrease it by 2

ARM assembly

Procedures / functions and the stack

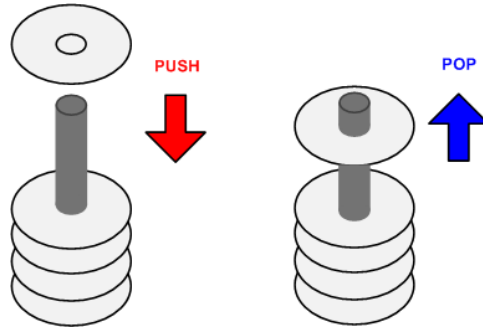


Using More Registers

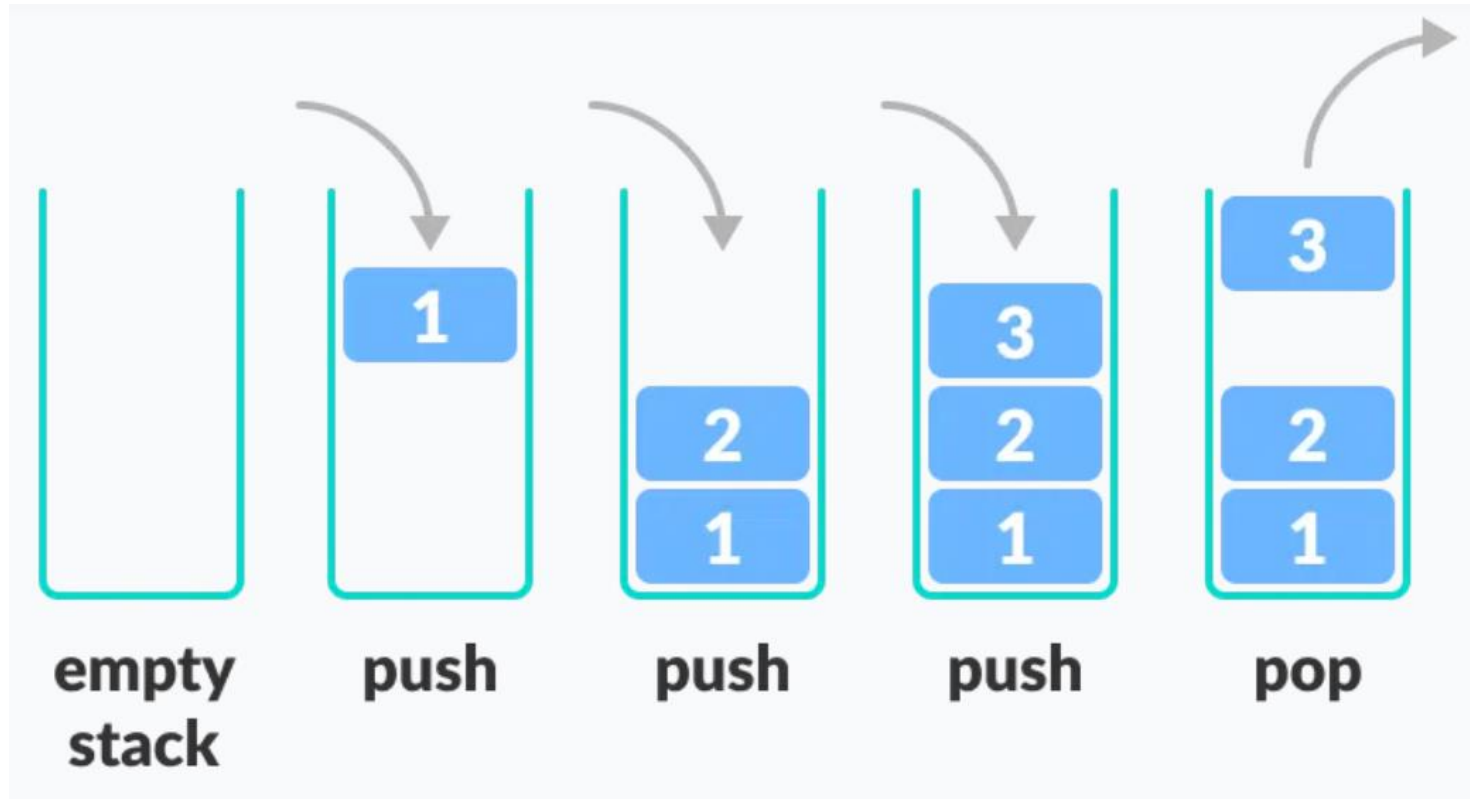
- What happens if we need more than four registers for a procedure?
- Any registers needed by the caller must be restored to the values that they contained before the procedure was invoked
- This situation is an example in which we need to spill registers to memory
- The ideal data structure for spilling registers is a **stack** - a LIFO queue. It needs a pointer to the most recently allocated address in the stack to show where the next procedure should place the registers to be spilled or where old register values exist
- The stack pointer (SP), which is just one of the registers, is adjusted by one **word** for each register that is saved or restored
- Stacks are so popular that they have their own buzzwords for transferring data to and from the stack: placing data onto the stack is called a **push**, and removing data from the stack is called a **pop**

Stack

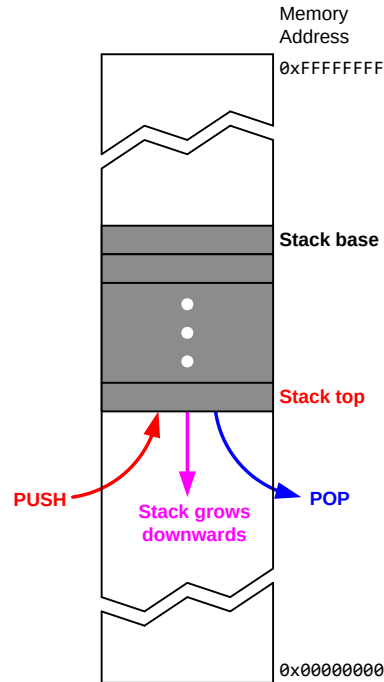
- A **Last-In-First-Out** memory model
- Only allow to access the most recently added item
 - Also called the top of the stack
- Key operations:
 - **push** (add item to stack)
 - **pop** (remove top item from stack)



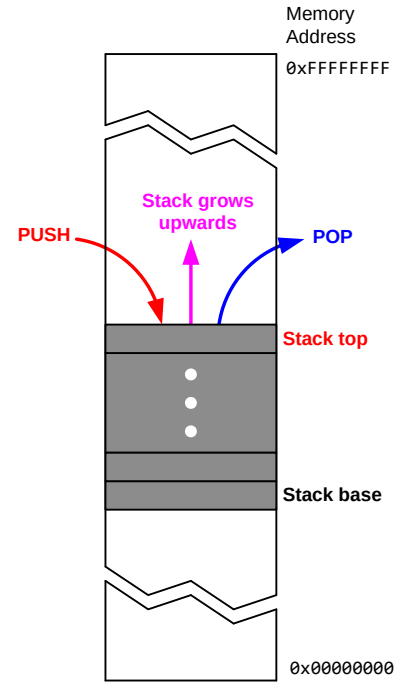
Stack



Stack Growth Convention: Ascending vs Descending



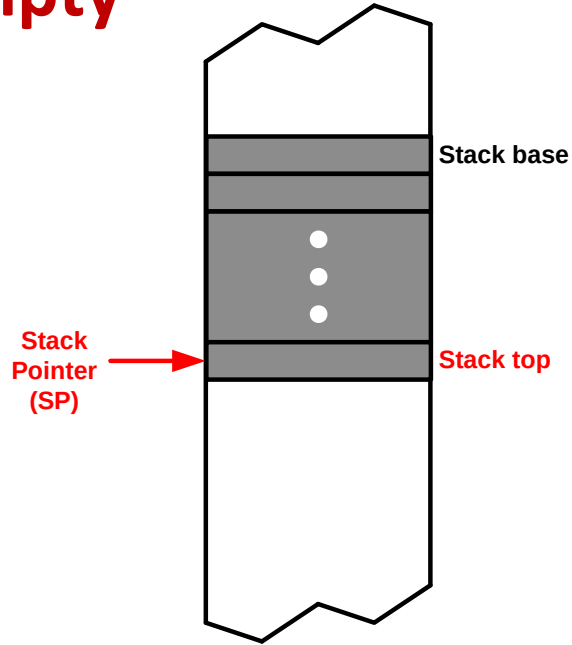
Descending stack: Stack grows towards low memory address



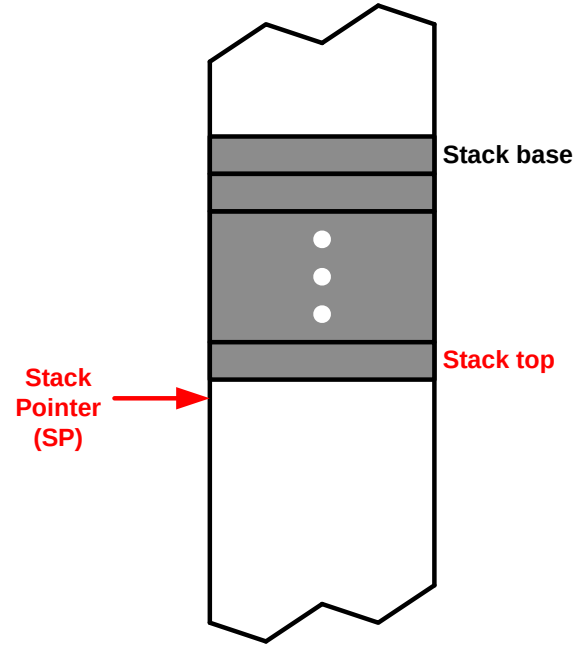
Ascending stack: Stack grows towards high memory address

Stack Growth Convention:

Full vs Empty



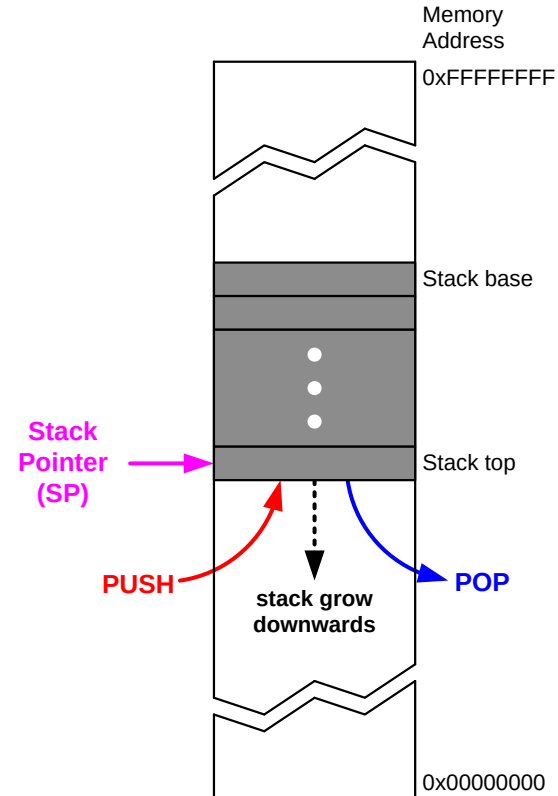
Full stack: SP points to the last item pushed onto the stack



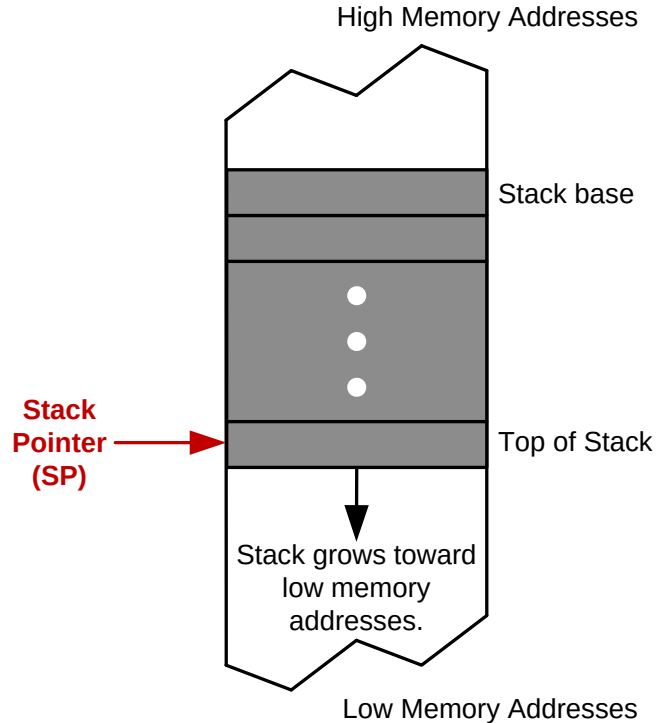
Empty stack: SP points to the next free space on the stack

Cortex-M Stack

- Cortex-M uses *full descending stack*
- Example:
PUSH/POP {r0,r6,r3}
- Stack pointer (SP, aka R13)
 - decremented on **PUSH**
 - $SP = SP - 4 * \text{\# of registers}$
 - incremented on **POP**
 - $SP = SP + 4 * \text{\# of registers}$
 - SP starts at 0x20000200 for STM32-Discovery by default (can be changed in startup.s)



Full Descending Stack



PUSH {register_list}
equivalent to:
STMDB SP!, {register_list}

DB: Decrement Before

POP {register_list}
equivalent to:
LDMIA SP!, {register_list}

IA: Increment After

Typical Usage of Stack

Why we need the stack?

- Saving the original contents of processor's registers at the beginning a subroutine (Contents are restored at the end of a subroutine)
- Storing local variables in a subroutine
- Passing extra arguments to a subroutine
- Saving processor's registers upon an interrupt

Stack

PUSH {Rd}

- $SP = SP - 4 \rightarrow$ descending stack
- $(*SP) = Rd \rightarrow$ full stack

Push multiple registers

They are equivalent.

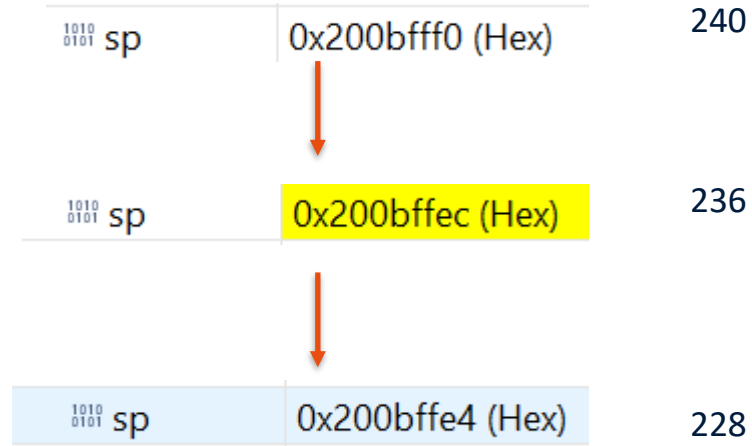
PUSH {r6, r7, r8}  **PUSH {r8, r7, r6}**  **PUSH {r8}**
PUSH {r7}
PUSH {r6}

- The order in which registers listed in the register list does not matter.
- When pushing multiple registers, these registers are automatically **sorted by name** and **the lowest-numbered register** is stored to the lowest memory address, *i.e.* **is stored last**.

Stack

Let's try in the IDE:

```
43 MOV r0, #5
44 MOV r1, #10
45 MOV r2, #15
46
47 push {r0}
48 MOV r0, #20
49 push {r0, r1}
```



200BFFE0	FC	04	00	08	14	00	00	00	0A	00	00	00	05	00	00	00
200BFFF0	28	1D	01	00	03	00	00	00	00	00	00	00	7F	04	00	08

Stack

POP {Rd}

- $Rd = (*SP) \rightarrow$ full stack
- $SP = SP + 4 \rightarrow$ Stack shrinks

Pop multiple registers

They are equivalent.

POP {r6, r7, r8} $\longleftrightarrow$ POP {r8, r7, r6} $\longleftrightarrow$
 POP {r6}
 POP {r7}
 POP {r8}

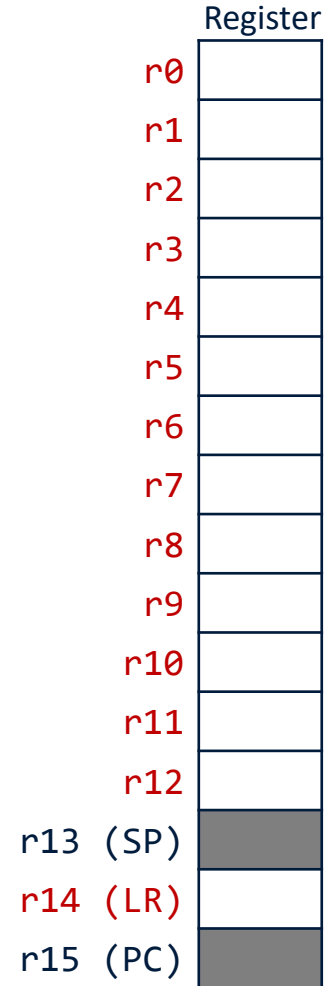
- The order in which registers listed in the register list does not matter.
- When popping multiple registers, these registers are automatically sorted by name and the lowest-numbered register is loaded from the lowest memory address, *i.e.* is loaded first.

Cortex-M Stack

- **PUSH {register list}**
- **POP {register list}**
 - Eligible registers for PUSH: r0 – r12, LR
 - Eligible registers for POP: r0 – r12, LR, PC
 - Order specified in the list does not matter
 - Largest-numbered register is always pushed first and popped last
 - Lowest-numbered register stored at lowest memory address

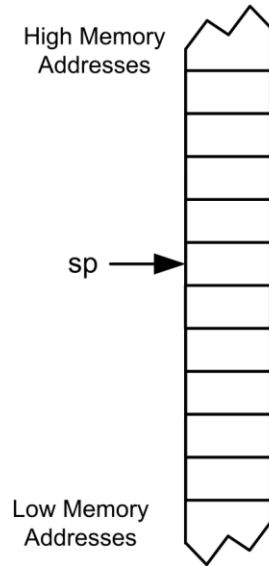
Instruction format for PUSH:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	1	0	0	1	0	1	1	0	1	(0)	M	(0)	register_list												



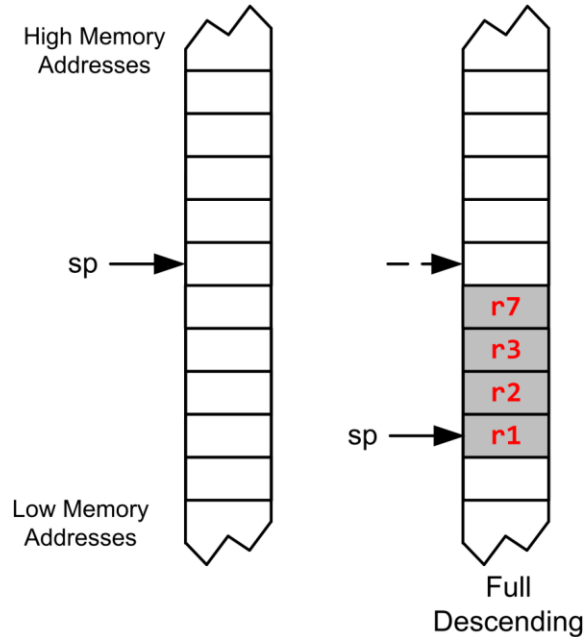
Full Descending Stack

PUSH {r3, r1, r7, r2}



Full Descending Stack

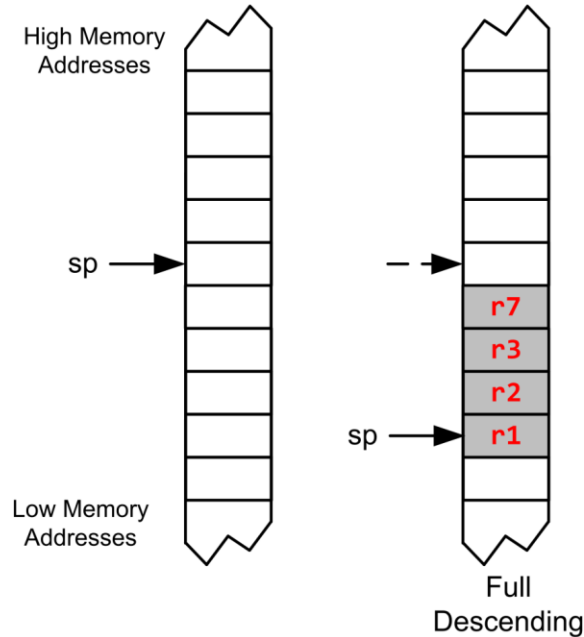
PUSH {r3, r1, r7, r2}



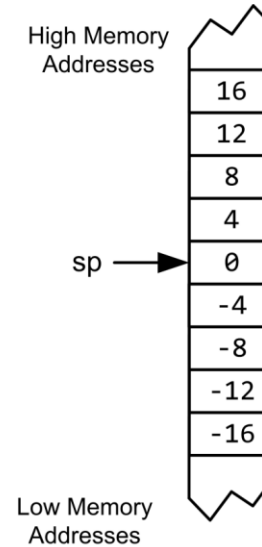
**Largest-numbered register
is pushed first.**

Full Descending Stack

PUSH {r3, r1, r7, r2}

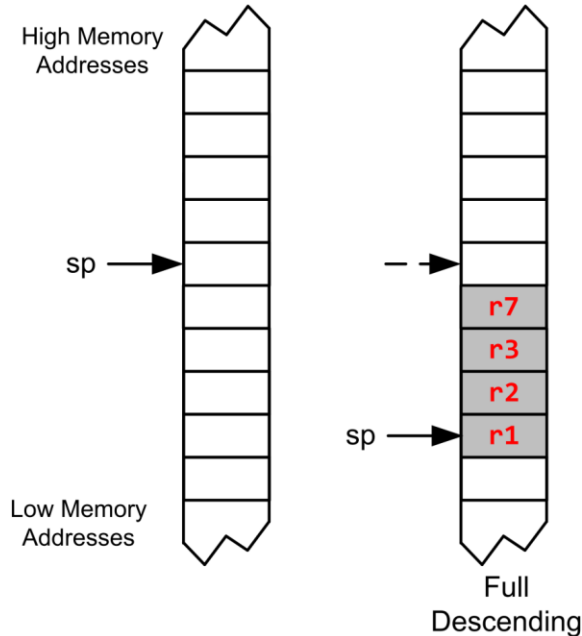


POP {r3, r1, r7, r2}



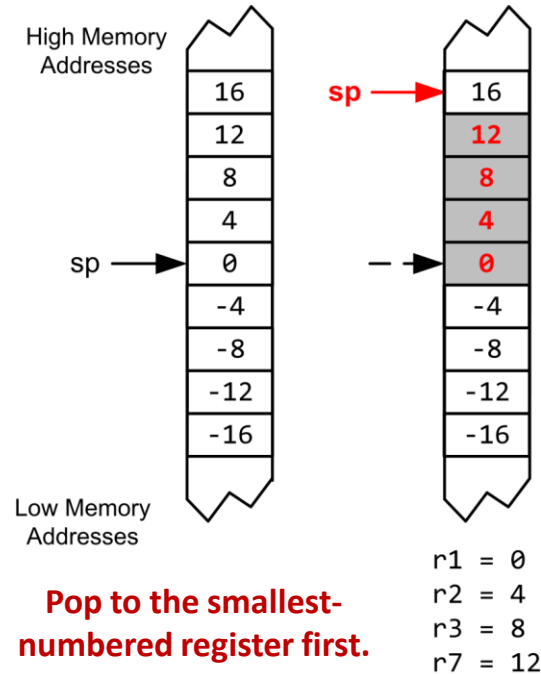
Full Descending Stack

PUSH {r3, r1, r7, r2}

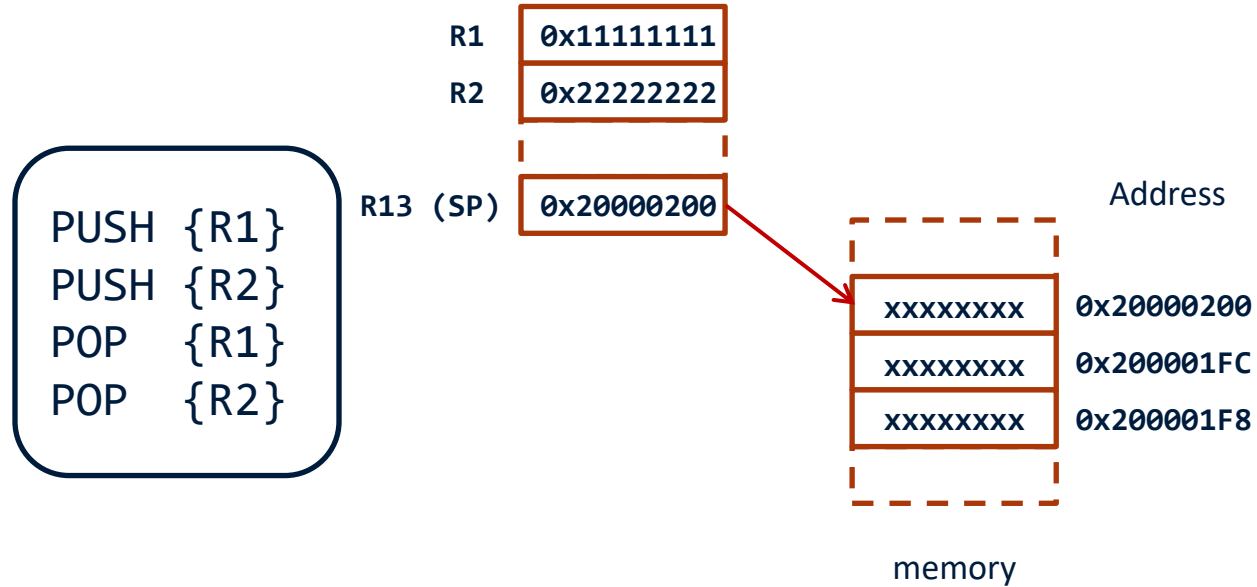


Largest-numbered register is pushed first but popped last.

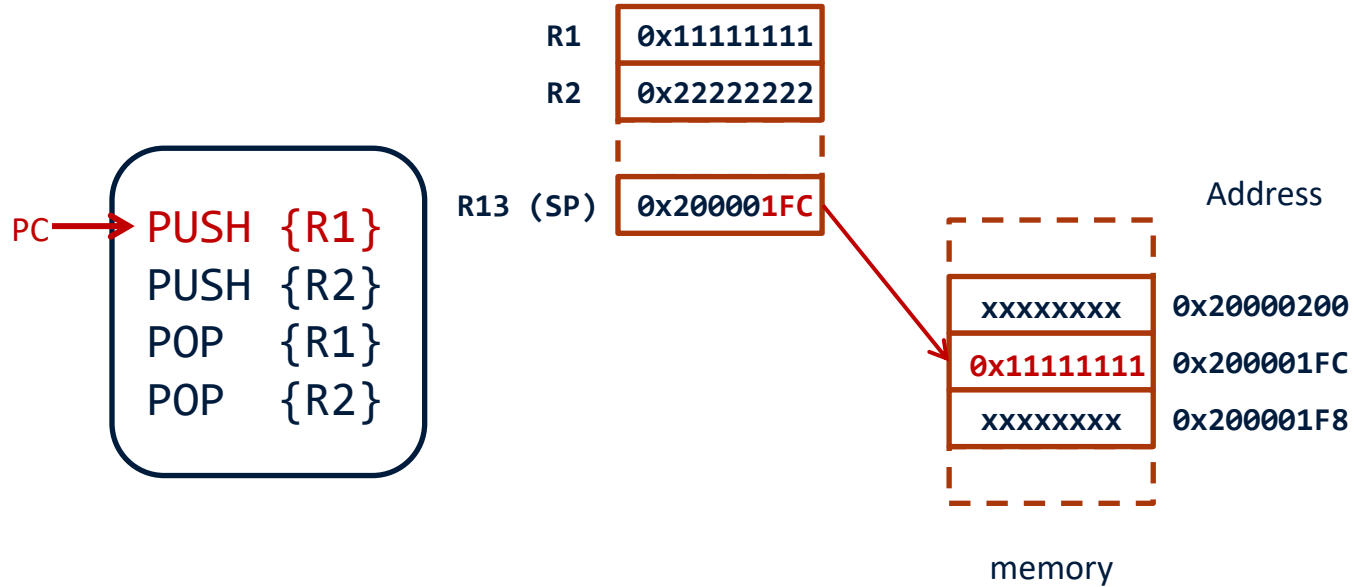
POP {r3, r1, r7, r2}



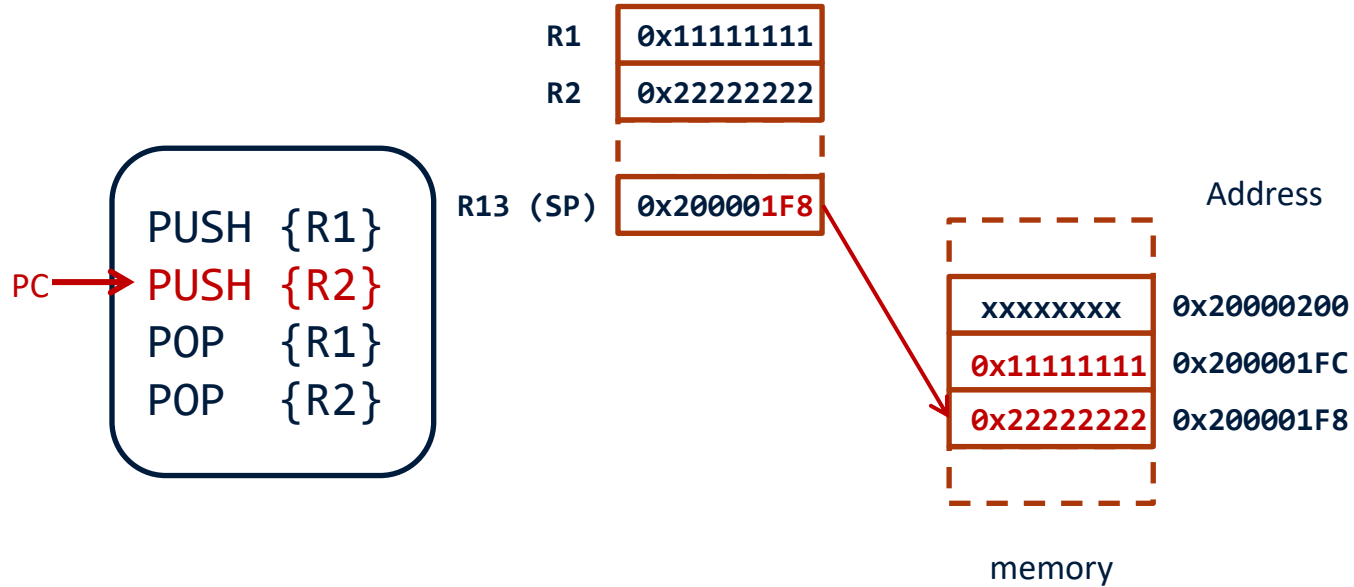
Example: swap R1 & R2



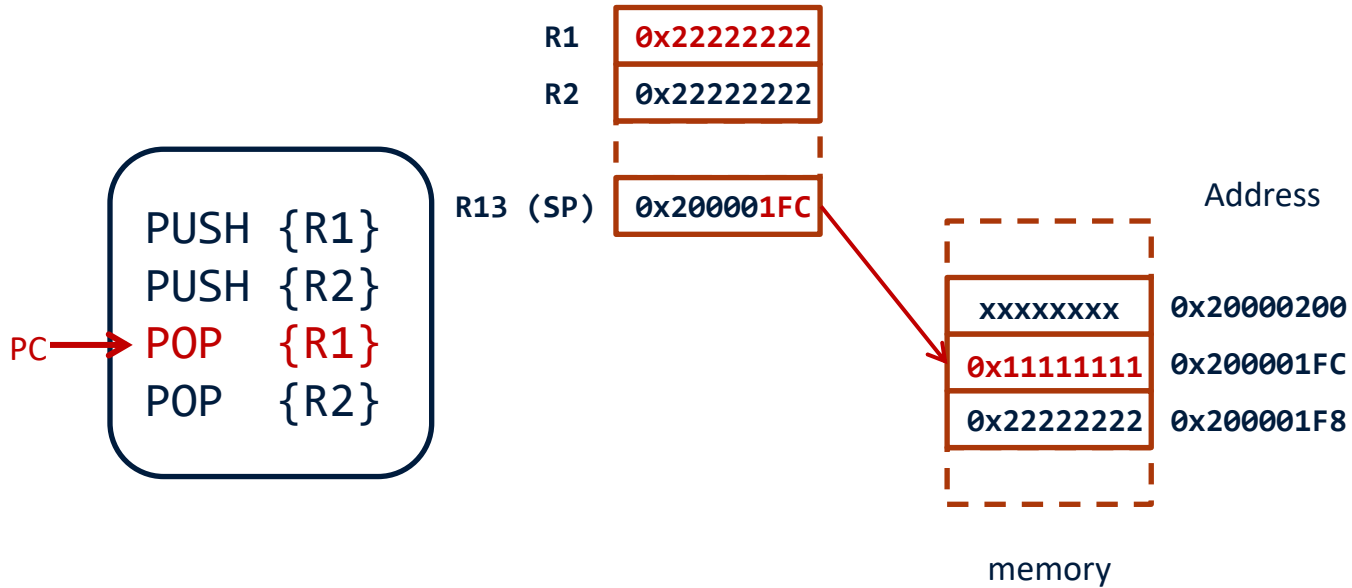
Example: swap R1 & R2



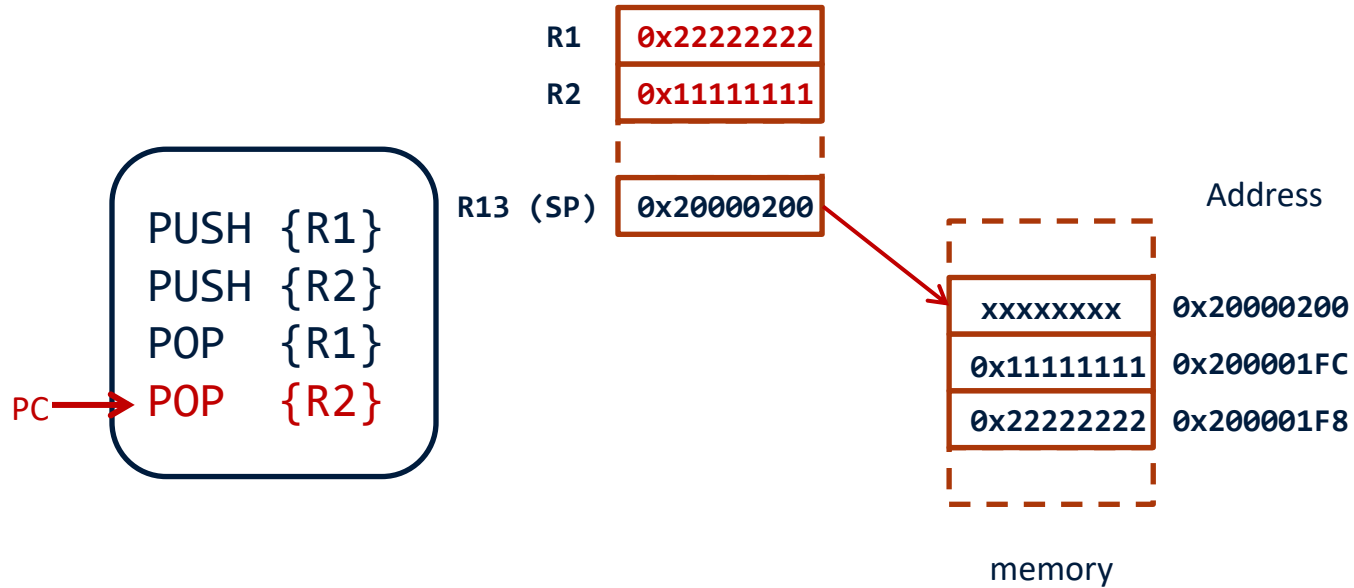
Example: swap R1 & R2



Example: swap R1 & R2



Example: swap R1 & R2



Stack

MOV R0, #99

PUSH {R0} // Push R0 to stack

MOV R0, #3

POP {R0} // Pop R0 from stack

BX LR

200BFFE0	BC04	0008	0000	0000	5D02	0008	9304	0008
200BFFF0	0000	0000	0000	0000	0000	0000	3F04	0008



200BFFE0	BC04	0008	0000	0000	5D02	0008	▲ 6300	▲ 0000
200BFFF0	0000	0000	0000	0000	0000	0000	3F04	0008



200BFFE0	BC04	0008	0000	0000	5D02	0008	6300	0000
200BFFF0	0000	0000	0000	0000	0000	0000	3F04	0008

1010
0101 sp

0x200bfff0



1010
0101 sp

0x200bffec



1010
0101 sp

0x200bfff0

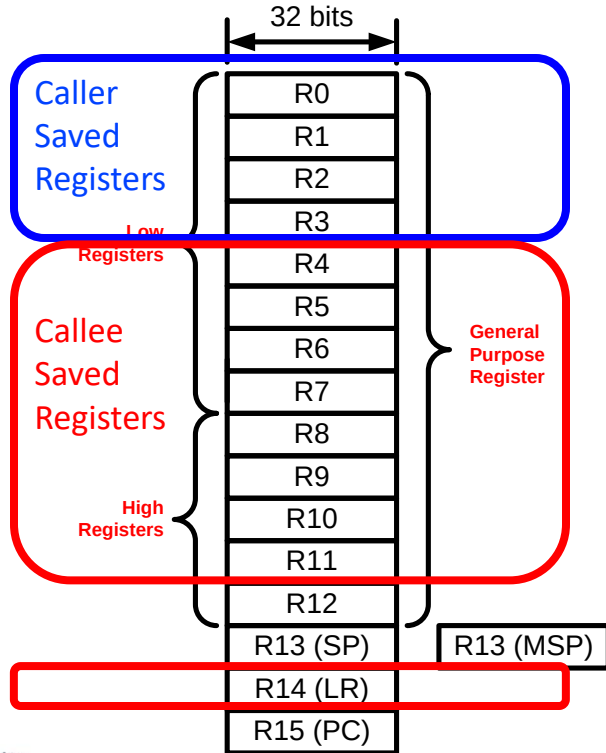
What to push to stack and when?

- The ARM Procedure Call Standard (APCS) defines a set of rules for procedure entry and exit so that
 - Object codes generated by different compilers can be linked together
 - Procedures can be called between high-level languages and assembly
- APCS defines:
 - Use of registers
 - Use of stack
 - Format of stack-based data structure
 - Mechanism for argument passing

ARM Procedure Call Standard

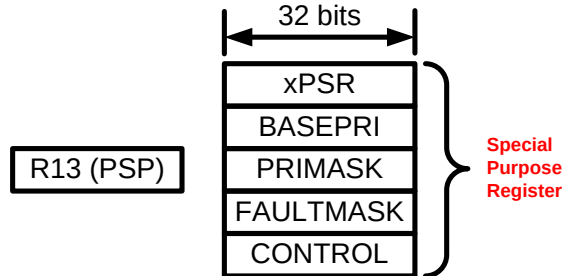
Register	Usage	Subroutine Preserved	Notes
r0	Argument 1 and return value	No	If return has 64 bits, then r0:r1 hold it. If argument 1 has 64 bits, r0:r1 hold it.
r1	Argument 2	No	
r2	Argument 3	No	If the return has 128 bits, r0-r3 hold it.
r3	Argument 4	No	If more than 4 arguments, use the stack
r4	General-purpose V1	Yes	Variable register 1 holds a local variable.
r5	General-purpose V2	Yes	Variable register 2 holds a local variable.
r6	General-purpose V3	Yes	Variable register 3 holds a local variable.
r7	General-purpose V4	Yes	Variable register 4 holds a local variable.
r8	General-purpose V5	Yes	Variable register 5 holds a local variable.
r9	Platform specific/V6	Yes	Usage is platform-dependent.
r10	General-purpose V7	Yes	Variable register 7 holds a local variable.
r11	General-purpose V8	Yes	Variable register 8 holds a local variable.
r12 (IP)	Intra-procedure-call register	No	It holds intermediate values between a procedure and the sub-procedure it calls.
r13 (SP)	Stack pointer	Yes	SP has to be the same after a subroutine has completed.
r14 (LR)	Link register	No	LR does not have to contain the same value after a subroutine has completed.
r15 (PC)	Program counter	N/A	Do not directly change PC

Callee Saved Registers vs Caller Saved Registers



- Callee can freely modify R0, R1, R2, and R3
- If caller expects their values are retained, caller should push them onto the stack before calling the callee

- **Caller expects these values are retained .**
- **If Callee modifies them, callee must restore their values upon leaving the function.**



What to push to stack?

- Argument passing:
 - The first four word arguments are passed through R0 to R3
 - Remaining parameters are pushed into stack in the reverse order
 - Procedures with less than four parameters are more effective
- Return values:
 - One word value in R0
 - A value of 2-4 words (R0-R1, R0-R2, R0-R3)

What to push to stack?

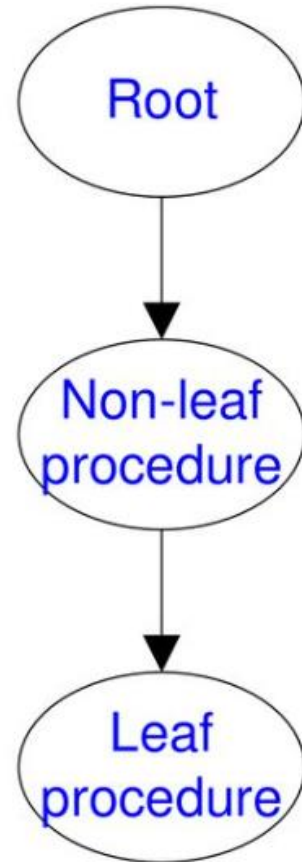
- R0-R3 are volatile, **caller-saved** registers
 - They are used to pass arguments to the function (R0-R3 for up to four arguments)
 - The result is returned in R0 (and sometimes R1 if the result is larger than one register can hold)
 - If the caller needs the values in these registers after the function call returns, it must save them to the stack before making the function call

What to push to stack?

- R4-R11, LR and SP are non-volatile, **callee-saved** registers.
 - If the called function uses them, it must push them to the stack and restore them before returning
 - These registers must retain their values after the function call is complete, so the callee is responsible for saving and restoring them if they are modified
 - If your function does not modify any of r4-r11 or lr, then you don't need to push anything
 - If the function modifies any of r4-r11, or uses lr for a return address, then you must push those to the stack at the beginning of the function and pop them before returning

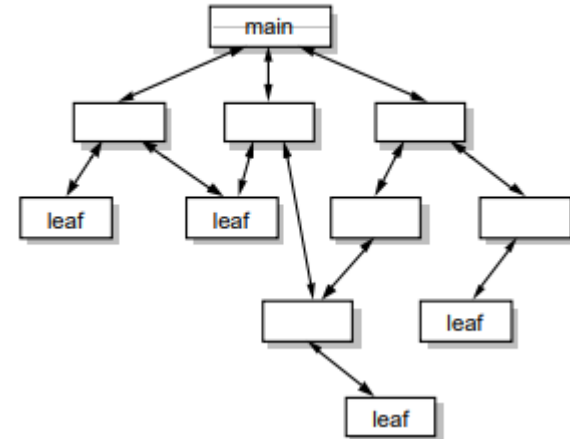
Procedures

- Root
 - Entry function of a program
 - Always a caller
- Non-leaf procedure
 - Both a caller and a callee
- Leaf procedure
 - Always a callee



What to push to stack?

- A simple leaf function with less than four parameters has the minimal overhead
- 50% of calls are to leaf functions



What to push to stack?

- A root procedure is always a **caller**
- A leaf procedure is always a **callee**
- **A non-leaf procedure is both a caller and a callee**
- A non-leaf function needs to manage LR more carefully because LR will be overwritten by the called function.
- Typically, a non-leaf function must:
 - Push LR to the stack at the start of the function because the return address will be overwritten when calling another function.
 - If callee-saved registers (R4-R11) are modified, they must also be pushed to the stack

Passing 4 Arguments

```
int32_t sum(int16_t a16, int16_t b16, int8_t c8, int32_t d32);
```

```
s = sum(1, 2, 3, 4);
```

Caller

```
MOVS r0, #1 ; a16
MOVS r1, #2 ; b16
MOVS r2, #3 ; c8
MOVS r3, #4 ; d32
BL    sum
```

Callee

```
sum PROC
    ADD r0, r0, r1 ; a16 + b16
    ADD r0, r0, r2 ; add c8
    ADD r0, r0, r3 ; add d32
    BX  LR          ; return
ENDP
```

Passing Extra Arguments via Stack

```
int32_t sum(int32_t a, int32_t b, int32_t c, int32_t d,  
            int32_t h, int32_t i, int32_t j, int32_t k);
```

```
s = sum(1, 2, 3, 4, 5, 6, 7, 8);
```

Caller

```
MOVS r0, #5  
MOVS r1, #6  
MOVS r2, #7  
MOVS r3, #8  
PUSH {r0, r1, r2, r3}  
MOVS r0, #1  
MOVS r1, #2  
MOVS r2, #3  
MOVS r3, #4  
BL    sum  
...  
POP {r0, r1, r2, r3}
```

Callee

```
sum PROC  
EXPORT sum  
ADD r0, r0, r1    ; add a + b  
ADD r0, r0, r2    ; add c  
ADD r0, r0, r3    ; add d  
LDRD r1,r2, [sp] ; r1=mem[sp],r2=mem[sp+4]  
ADD r0, r0, r1    ; add h  
ADD r0, r0, r2    ; add i  
LDRD r1,r2, [sp, #8] ;  
r1=mem[sp+8],r2=mem[sp+12]  
ADD r0, r0, r1    ; add j  
ADD r0, r0, r2    ; add k  
BX    LR  
ENDP
```

LDRD: Load Register Doubleword

Passing Extra Arguments via Stack

```
int32_t sum(int32_t a, int32_t b, int32_t c, int32_t d,  
            int32_t h, int32_t i, int32_t j, int32_t k);
```

```
s = sum(1, 2, 3, 4, 5, 6, 7, 8);
```

Caller

```
MOVS r0, #5  
MOVS r1, #6  
MOVS r2, #7  
MOVS r3, #8  
PUSH {r0, r1, r2, r3}  
MOVS r0, #1  
MOVS r1, #2  
MOVS r2, #3  
MOVS r3, #4  
BL    sum  
...  
POP {r0, r1, r2, r3}
```

Callee

```
sum PROC  
EXPORT sum  
PUSH {r5, r6, lr}  
ADD r0, r0, r1    ; add a + b  
ADD r0, r0, r2    ; add c  
ADD r0, r0, r3    ; add d  
LDRD r5,r6, [sp, #12]  
;r5=mem[sp+12],r6=mem[sp+16]  
ADD r0, r0, r5    ; add h  
ADD r0, r0, r6    ; add i  
LDRD r5,r6, [sp, #20]  
;r5=mem[sp+20],r6=mem[sp+24]  
ADD r0, r0, r5    ; add j  
ADD r0, r0, r6    ; add k  
POP {r5, r6, pc}  
ENDP
```

Passing Extra Arguments via Stack

```
int32_t sum(int32_t a, int32_t b, int32_t c, int32_t d,  
            int32_t h, int32_t i, int32_t j, int32_t k);
```

```
s = sum(1, 2, 3, 4, 5, 6, 7, 8);
```

Caller

```
MOVS r0, #5  
MOVS r1, #6  
MOVS r2, #7  
MOVS r3, #8  
PUSH {r0, r1, r2, r3}  
MOVS r0, #1  
MOVS r1, #2  
MOVS r2, #3  
MOVS r3, #4  
BL    sum  
...  
POP {r0, r1, r2, r3}
```

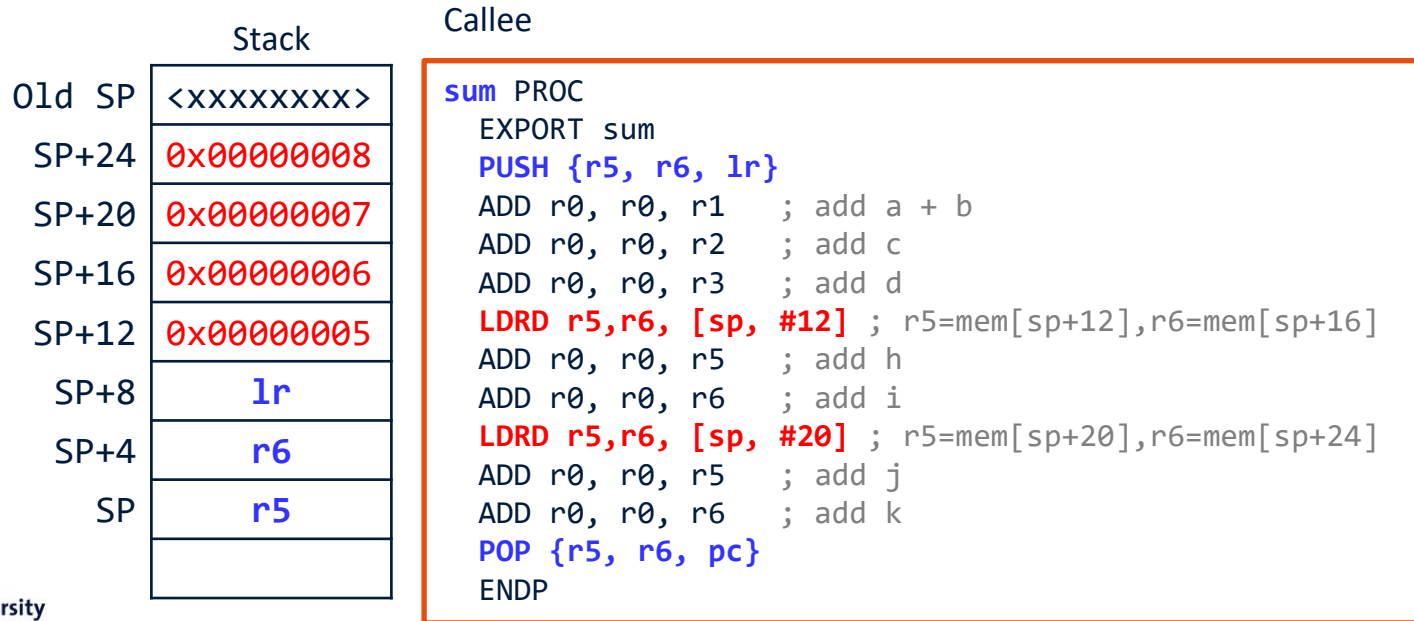
Stack

Old SP	<xxxxxxxx>
SP+12	0x00000008
SP+8	0x00000007
SP+4	0x00000006
SP	0x00000005

Passing Extra Arguments via Stack

```
int32_t sum(int32_t a, int32_t b, int32_t c, int32_t d,  
            int32_t h, int32_t i, int32_t j, int32_t k);
```

```
s = sum(1, 2, 3, 4, 5, 6, 7, 8);
```



Summary

- How to call a subroutine?
 - Branch with link: **BL subroutine**
- How to return the control back to the caller?
 - Branch and exchange: **BX LR**
- How to pass arguments into a subroutine?
 - Each 8-, 16- or 32-bit variables is passed via r0, r1, r2, r3
 - Extra parameters are passed via stack
- How to return a value in a subroutine?
 - Value is returned in r0